



COMPONENT COMMUNICATION (Child to Parent)



codewithrutu_09

✓ Child → Parent Communication?

In real-world apps, the child component often needs to:

📡 Send user actions back (e.g. button clicks)

📡 Inform the parent of a value change
📡 Trigger logic in the parent (like removing an item)

➡ We use `@Output()` and `EventEmitter` for this.

✓ @Output() + EventEmitter

In the child component:



app.component.ts

```
@Output() notify = new EventEmitter<string>();
```

@Output() marks a property as an event source

EventEmitter emits the event and optional data to the parent

✓ Emitting an Event from Child

Trigger the event using `.emit()`:

```
app.component.ts

sendMessage() {
  this.notify.emit('Message from child!');
}
```

💡 You can emit any data (string, number, object, etc.)

✓ Parent Listens to Child Event

In the parent component HTML:

```
app.component.html  
  
<app-child (notify)="onNotify($event)"></app-child>
```

In the parent TS file:

```
app.component.ts  
  
onNotify(message: string) {  
  console.log('Received:', message);  
}
```

Parent listens using () syntax (event binding)

✓ Real-World Example

Child Component:

```
app.component.ts

@Output() deleteItem = new EventEmitter<number>();

delete(id: number) {
  this.deleteItem.emit(id);
}
```

Parent Component:

```
app.component.html

<app-todo (deleteItem)="removeItem($event)"></app-todo>
```

```
app.component.ts

removeItem(id: number) {
  this.todoList = this.todoList.filter(item => item.id !== id);
}
```

Output Property Best Practices

Best Practices:

- Use meaningful names for output events (onSave, itemDeleted, etc.)
- Clearly define what type of data you're emitting
- Keep emit logic simple and focused
- Use @Output() only when parent needs to know something happened

✓ Emit Custom Objects

You can also emit structured objects:

```
app.component.ts

@Output() productEvent = new EventEmitter<{ id: number,
name: string }>();

productEvent.emit({ id: 1, name: 'Laptop' });
```

In parent:

```
app.component.ts

(productEvent)="handleProduct($event)"
```

Ideal for sending detailed data

Key Takeaways

- Use `@Output()` in child to define an event
- Use `.emit()` to send data from child
- Parent listens with `()` and `$event`
- Use for actions like form submissions, button clicks, etc.
- Keep communication clean and meaningful



codewithrutu_09



Like this post!



Share with your friends



Save it for later



**Tell us what you think in
comment**



codewithrutu_09



Rutuja Shinde